```
# kubectl get pods -A
NAMESPACE     NAME                                       READY   STATUS    RESTARTS   AGE
kube-system   calico-kube-controllers-6b77fff45-xwdlh    1/1     Running   0          7m35s
kube-system   calico-node-whskl                          1/1     Running   0          7m36s
kube-system   calico-typha-db644df6c-2nvmz               1/1     Running   0          7m36s
kube-system   coredns-64897985d-dlztb                    1/1     Running   0          8m32s
kube-system   etcd-calico-demo                           1/1     Running   0          8m44s
kube-system   kube-apiserver-calico-demo                 1/1     Running   0          8m47s
kube-system   kube-controller-manager-calico-demo        1/1     Running   0          8m44s
kube-system   kube-proxy-jb5qn                           1/1     Running   0          8m32s
kube-system   kube-scheduler-calico-demo                 1/1     Running   0          8m47s
kube-system   storage-provisioner                        1/1     Running   0          8m41s
```

Figure 1: Details of running pods

these in the official documentation of Kubernetes.

Calico is a popular open source CNI plugin available for Kubernetes. Created and maintained by Tigera, Calico focuses on the performance, flexibility and security of the network. It offers scalable networking functions using the standard L3 approach.

There are three major advantages of using Calico.

Scalability: Calico is built on a fully distributed, layer 3 based scale-out architecture; so it scales from a single developer laptop to large enterprise deployments. Calico configures a layer 3 network that uses the BGP routing protocol to route packets between hosts. This means that packets do not need to be wrapped in an extra layer of encapsulation when moving between hosts. The BGP routing mechanism can direct packets natively without the extra step of wrapping traffic in an additional layer of traffic.

Debugging: Calico relies on an IP layer and is relatively easy to debug with existing tools, an important aspect for enterprise security.

Micro-segmentation support: The plugin makes it possible for administrators or end users to define network policies between multiple pods. Network policies in Kubernetes are essentially firewalls for pods. Calico network policies extend the functionalities of Kubernetes network policies. By default, pods are accessible from anywhere with no protection. With Calico network policies, we can control which pods can send and receive traffic and also manage security within the network using zero trust networking architecture. By leveraging the native Linux kernel, Calico users can also utilise existing network tools, including IP-tables, to perform high-level micro-segmentation.

A demo of how Calico works

For the sake of simplicity, we will use minikube for the demo. minikube quickly sets up a local Kubernetes cluster on macOS, Linux, and Windows. Please refer to minikube documentation for the installation. We will install the Calico CNI plugin on a cluster created by minikube.

Let us use minikube to start a cluster by giving the following command:

```
$ minikube start --network-plugin=cni --cni=calico
```

Once the cluster is up and running, you can install Calico *(https://docs.projectcalico.org/manifests/calico-typha.yaml):*

```
$ kubectl apply -f
$ kubectl get pods -A
```

Figure 1 shows the output of the above command.

Calico provides a command line tool called *calicoctl*, which is required to use many of Calico's features. It can be used to manage Calico network policies and configurations. You can install *calicoctl* as follows *(https://docs.projectcalico.org/manifests/calicoctl.yaml):*

```
$ kubectl apply -f
serviceaccount/calicoctl created
pod/calicoctl created
clusterrole.rbac.authorization.k8s.io/calicoctl created
clusterrolebinding.rbac.authorization.k8s.io/calicoctl
created
```

You can use *calicoctl* as shown below:

```
#kubectl exec -ti -n kube-system calicoctl -- /calicoctl get
profiles
NAME
projectcalico-default-allow
kns.default
kns.kube-node-lease
kns.kube-public
```